

Nonlinear Conjugate Gradient With Perturbed Heavy-Ball Saddle Escape

Benjamin Séguin

Université Paris Dauphine-PSL

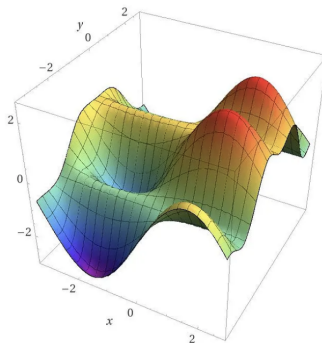
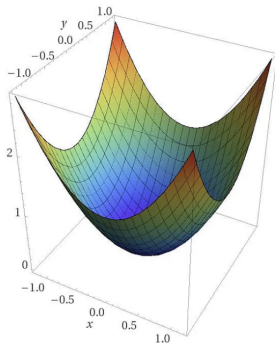
Supervised by Clément W. Royer

Master's thesis defence, 22nd September 2026

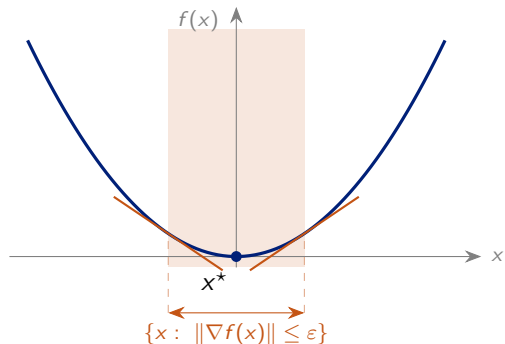
$$\min_{x \in \mathbb{R}^n} f(x)$$

$$\min_{x \in \mathbb{R}^n} f(x)$$

Algorithms have different guarantees depending on whether f is convex or nonconvex.



Convex optimization: a small gradient is enough



the tangents at the edges of the band have slope $\pm\varepsilon$

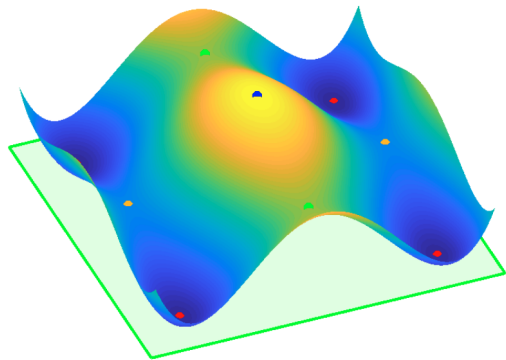
Convex functions have only one global minimum at which $\nabla f(x) = 0$.

We look for an ε -first-order stationary point (FOSP):

$$\|\nabla f(x)\| \leq \varepsilon.$$

As $\varepsilon \rightarrow 0$ we certify a local minimum, and every local minimum is **global**.

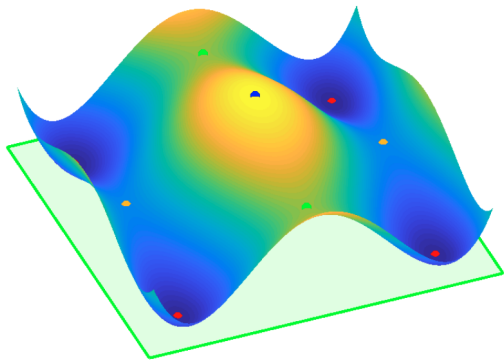
Nonconvex: a small gradient is not enough



Wright & Ma '22

There are **many** points with $\nabla f(x) = 0$, and not all of them are minima. The others are local maxima and saddle points.

Nonconvex: a small gradient is not enough



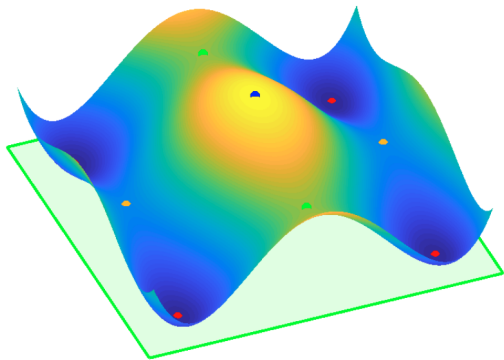
Wright & Ma '22

There are **many** points with $\nabla f(x) = 0$, and not all of them are minima. The others are local maxima and saddle points.

We therefore ask for an $(\varepsilon_g, \varepsilon_H)$ -second-order stationary point (SOSP):

$$\|\nabla f(x)\| \leq \varepsilon_g \quad \text{and} \quad \lambda_{\min}(\nabla^2 f(x)) \geq -\varepsilon_H.$$

Nonconvex: a small gradient is not enough



Wright & Ma '22

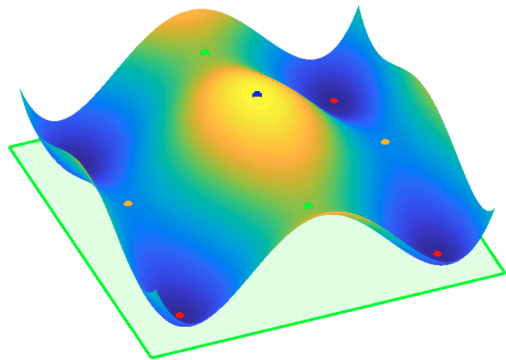
There are **many** points with $\nabla f(x) = 0$, and not all of them are minima. The others are local maxima and saddle points.

We therefore ask for an $(\varepsilon_g, \varepsilon_H)$ -**second-order stationary point (SOSP)**:

$$\|\nabla f(x)\| \leq \varepsilon_g \quad \text{and} \quad \lambda_{\min}(\nabla^2 f(x)) \geq -\varepsilon_H.$$

This rules out **nondegenerate** local maxima and saddle points, and is the standard target in nonconvex optimization. We use the special case $\varepsilon_H = \sqrt{\varepsilon_g}$ throughout.

Nonconvex: a small gradient is not enough



Wright & Ma '22

There are **many** points with $\nabla f(x) = 0$, and not all of them are minima. The others are local maxima and saddle points.

We therefore ask for an $(\varepsilon_g, \varepsilon_H)$ -second-order stationary point (SOSP):

$$\|\nabla f(x)\| \leq \varepsilon_g \quad \text{and} \quad \lambda_{\min}(\nabla^2 f(x)) \geq -\varepsilon_H.$$

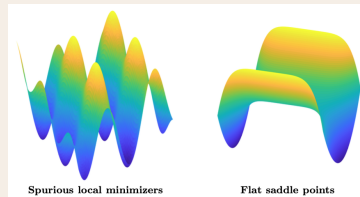
This rules out **nondegenerate** local maxima and saddle points, and is the standard target in nonconvex optimization. We use the special case $\varepsilon_H = \sqrt{\varepsilon_g}$ throughout.

Certifying an actual local, let alone global, minimum is **NP-hard** without further assumptions on the landscape. (Murty & Kabadi '87)

Nonconvex optimization: a matter of landscape

Bad instances

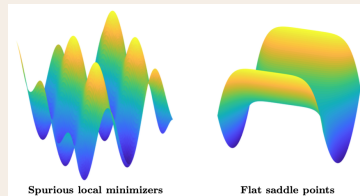
- Local, non-global minima.
- High-order saddle points.



Nonconvex optimization: a matter of landscape

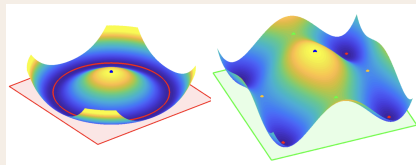
Bad instances

- Local, non-global minima.
- High-order saddle points.



Nice instances

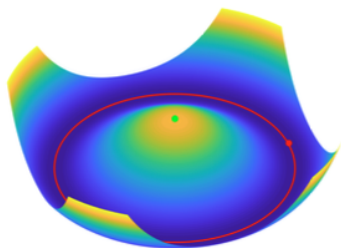
- All local minima are global.
- Strict (non-flat) saddle points.



Figures: J. Wright and Y. Ma, *High-Dimensional Data Analysis with Low-Dimensional Models*, 2022.

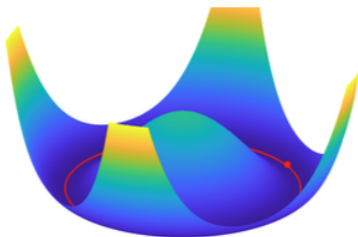
Generalized phase retrieval

$$\min_x \frac{1}{2} \|y^2 - |Ax|^2\|_2^2$$



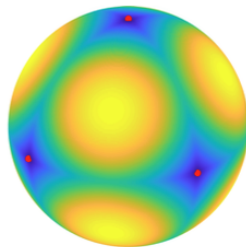
Matrix recovery

$$\min_{U,V} \mathcal{L}(Y - \mathcal{A}[UV^*]) + \rho(U, V)$$



Dictionary learning

$$\min_{A \in \mathcal{A}, X} \frac{1}{2} \|Y - AX\|_F^2 + \lambda \|X\|_1$$



Pictures: J. Wright and Y. Ma, High-Dimensional Data Analysis with Low-Dimensional Models, 2022.

Escaping saddle points cheaply

- Do it with **gradients only**, no Hessian.
- Do it for **nonlinear conjugate gradient (NCG)**, a practical method with limited theory in the nonconvex setting.
- Prove a **complexity guarantee**: how many iterations, in the worst case.

What this work is about

Escaping saddle points cheaply

- Do it with **gradients only**, no Hessian.
- Do it for **nonlinear conjugate gradient (NCG)**, a practical method with limited theory in the nonconvex setting.
- Prove a **complexity guarantee**: how many iterations, in the worst case.

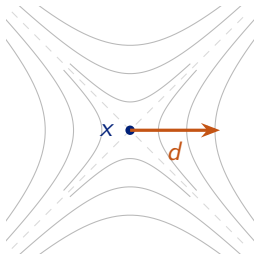
Main result

An algorithm that returns an $(\varepsilon_g, \varepsilon_H)$ -SOSP with high probability. To our knowledge, the first such guarantee for a method based on NCG.

1. How do we escape a saddle point?
2. Conjugate gradient and heavy-ball
3. The method and the result
4. Does it work?
5. Conclusion

1. How do we escape a saddle point?
2. Conjugate gradient and heavy-ball
3. The method and the result
4. Does it work?
5. Conclusion

Option 1: look at the curvature



level sets near a strict saddle

Negative curvature methods read the Hessian and step **along the direction of most negative curvature**.

- Best known rates: $O(\max\{\varepsilon_g^{-3/2}, \varepsilon_H^{-3}\})$, and this is sharp.

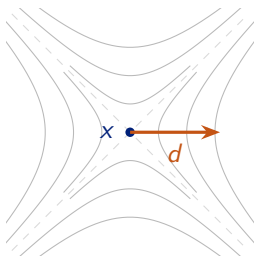
The Hessian $\nabla^2 f(x)$ collects the second derivatives and gives the local curvature:

$$f(x + d) \approx f(x) + \nabla f(x)^\top d + \frac{1}{2} d^\top \nabla^2 f(x) d.$$

d is a direction of **negative curvature** at x if

$$d^\top \nabla^2 f(x) d \leq -\varepsilon_H \|d\|^2.$$

Option 1: look at the curvature



level sets near a strict saddle

The Hessian $\nabla^2 f(x)$ collects the second derivatives and gives the local curvature:

$$f(x + d) \approx f(x) + \nabla f(x)^\top d + \frac{1}{2} d^\top \nabla^2 f(x) d.$$

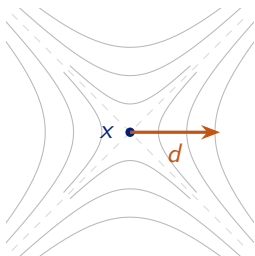
d is a direction of **negative curvature** at x if

$$d^\top \nabla^2 f(x) d \leq -\varepsilon_H \|d\|^2.$$

Negative curvature methods read the Hessian and step **along the direction of most negative curvature**.

- Best known rates: $O(\max\{\varepsilon_g^{-3/2}, \varepsilon_H^{-3}\})$, and this is sharp.
- But curvature is queried **at every iteration**, and each query costs several Hessian-vector products even in its cheapest form.

Option 1: look at the curvature



level sets near a strict saddle

The Hessian $\nabla^2 f(x)$ collects the second derivatives and gives the local curvature:

$$f(x + d) \approx f(x) + \nabla f(x)^\top d + \frac{1}{2} d^\top \nabla^2 f(x) d.$$

d is a direction of **negative curvature** at x if

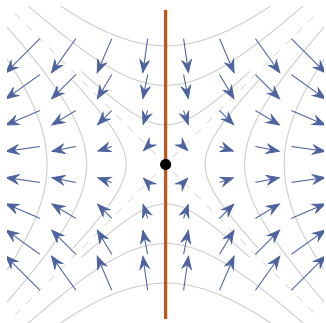
$$d^\top \nabla^2 f(x) d \leq -\varepsilon_H \|d\|^2.$$

Negative curvature methods read the Hessian and step **along the direction of most negative curvature**.

- Best known rates: $O(\max\{\varepsilon_g^{-3/2}, \varepsilon_H^{-3}\})$, and this is sharp.
- But curvature is queried **at every iteration**, and each query costs several Hessian-vector products even in its cheapest form.

In high dimension, that is what one would like to avoid. Nesterov & Polyak '06; Cartis et al. '11; Royer et al. '20; Curtis et al. '21.

Option 2: hope that gradient descent escapes fast

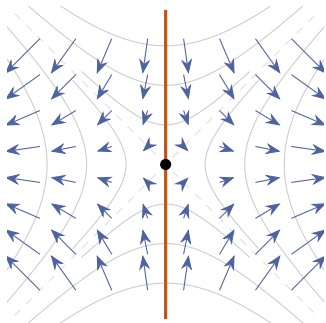


the initializations that get stuck

The gradient flow near a saddle pushes almost every point away. Only the initialisations on a lower-dimensional set converge to it.

- GD avoids strict saddles from almost every starting point.

Option 2: hope that gradient descent escapes fast

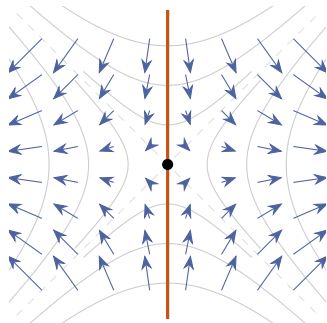


the initializations that get stuck

The gradient flow near a saddle pushes almost every point away. Only the initialisations on a lower-dimensional set converge to it.

- GD avoids strict saddles from almost every starting point.
- But the guarantee is asymptotic: no bound on how long it takes.

Option 2: hope that gradient descent escapes fast



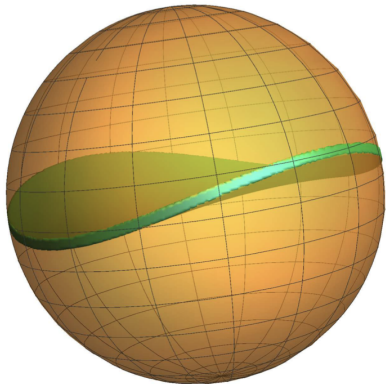
the initializations that get stuck

The gradient flow near a saddle pushes almost every point away. Only the initialisations on a lower-dimensional set converge to it.

- GD avoids strict saddles **from almost every starting point**.
- But the guarantee is **asymptotic**: no bound on how long it takes.
- And there are functions on which it takes **exponentially many** iterations to get away.

Lee et al. '16, '19; Muşat & Boumal '25; Du et al. '17.

Option 3: add a little noise

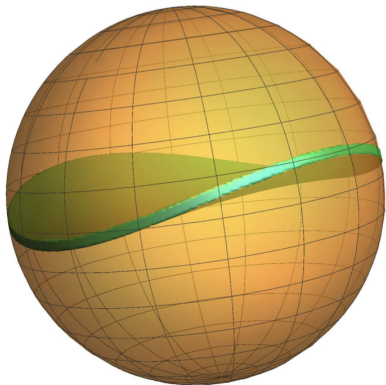


the ball of perturbations

When the gradient becomes small, **perturb the iterate** inside a small ball and keep going for a fixed number of steps.

- The perturbations that fail to escape form a **thin pancake**.

Option 3: add a little noise

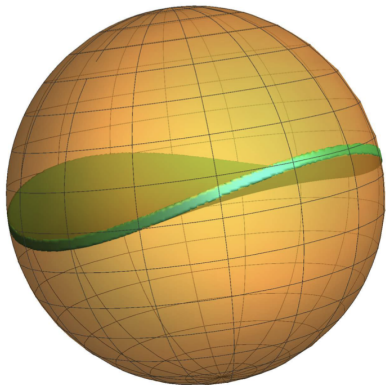


the ball of perturbations

When the gradient becomes small, **perturb the iterate** inside a small ball and keep going for a fixed number of steps.

- The perturbations that fail to escape form a **thin pancake**.
- Perturbed gradient descent (PGD) and perturbed accelerated gradient descent (PAGD) come from this framework.

Option 3: add a little noise



the ball of perturbations

When the gradient becomes small, **perturb the iterate** inside a small ball and keep going for a fixed number of steps.

- The perturbations that fail to escape form a **thin pancake**.
- Perturbed gradient descent (PGD) and perturbed accelerated gradient descent (PAGD) come from this framework.
- Result: an $(\varepsilon_g, \varepsilon_H)$ -SOSP in $\tilde{O}(\varepsilon_g^{-2})$ iterations for PGD, $\tilde{O}(\varepsilon_g^{-7/4})$ for PAGD.

Picture after Jin, Ge, Netrapalli, Kakade & Jordan '17; Jin, Netrapalli & Jordan '18.

Recall that a SOSP is defined as a point x such that:

$$\|\nabla f(x)\| \leq \varepsilon_g \quad \text{and} \quad \lambda_{\min}(\nabla^2 f(x)) \geq -\varepsilon_H.$$

In the table below, $\varepsilon_H = \sqrt{\varepsilon_g}$

	needs curvature?	reaches an $(\varepsilon_g, \varepsilon_H)$ -SOSP?	rate
Neg curvature	yes	yes	$\varepsilon_g^{-3/2}$
GD	no	asymptotically, no rate	—
PGD	no	yes, w.h.p.	$\tilde{O}(\varepsilon_g^{-2})$
PAGD	no	yes, w.h.p.	$\tilde{O}(\varepsilon_g^{-7/4})$

Recall that a SOSP is defined as a point x such that:

$$\|\nabla f(x)\| \leq \varepsilon_g \quad \text{and} \quad \lambda_{\min}(\nabla^2 f(x)) \geq -\varepsilon_H.$$

In the table below, $\varepsilon_H = \sqrt{\varepsilon_g}$

	needs curvature?	reaches an $(\varepsilon_g, \varepsilon_H)$ -SOSP?	rate
Neg curvature	yes	yes	$\varepsilon_g^{-3/2}$
GD	no	asymptotically, no rate	—
PGD	no	yes, w.h.p.	$\tilde{O}(\varepsilon_g^{-2})$
PAGD	no	yes, w.h.p.	$\tilde{O}(\varepsilon_g^{-7/4})$
NCG	no	unknown	—

1. How do we escape a saddle point?
2. Conjugate gradient and heavy-ball
3. The method and the result
4. Does it work?
5. Conclusion

Throughout, $g_k := \nabla f(x_k)$.

Heavy-ball

$$x_{k+1} = x_k - \alpha g_k + \gamma (x_k - x_{k-1}).$$

- Most classical momentum technique.
- $\gamma = 0 \rightarrow$ recover GD.
- Reuses the previous displacement.
- α and γ are **constants**, fixed in advance.

Conjugate gradient and heavy-ball

Throughout, $g_k := \nabla f(x_k)$.

Heavy-ball

$$x_{k+1} = x_k - \alpha g_k + \gamma (x_k - x_{k-1}).$$

- Most classical momentum technique.
- $\gamma = 0 \rightarrow$ recover GD.
- Reuses the previous displacement.
- α and γ are **constants**, fixed in advance.

Nonlinear conjugate gradient

$$x_{k+1} = x_k + \alpha_k d_k, \quad d_{k+1} = -g_{k+1} + \beta_{k+1} d_k.$$

- α_k from an Armijo line search.
- β_{k+1} a **nonlinear** function of two gradients:

$$\beta_{k+1}^{\text{PRP}^+} = \max \left\{ \frac{g_{k+1}^\top (g_{k+1} - g_k)}{\|g_k\|^2}, 0 \right\}.$$

Conjugate gradient and heavy-ball

Throughout, $g_k := \nabla f(x_k)$.

Heavy-ball

$$x_{k+1} = x_k - \alpha g_k + \gamma (x_k - x_{k-1}).$$

- Most classical momentum technique.
- $\gamma = 0 \rightarrow$ recover GD.
- Reuses the previous displacement.
- α and γ are **constants**, fixed in advance.

Nonlinear conjugate gradient

$$x_{k+1} = x_k + \alpha_k d_k, \quad d_{k+1} = -g_{k+1} + \beta_{k+1} d_k.$$

- α_k from an Armijo line search.
- β_{k+1} a **nonlinear** function of two gradients:

$$\beta_{k+1}^{\text{PRP}^+} = \max \left\{ \frac{g_{k+1}^\top (g_{k+1} - g_k)}{\|g_k\|^2}, 0 \right\}.$$

Both reuse past information at the cost of **one gradient per iteration**.

Polyak '64; Fletcher & Reeves '64; Polak–Ribière–Polyak '69.

Nonlinear conjugate gradient has thin theoretical guarantees

d_{k+1} may not be a descent direction, so implementations **restart**: throw the direction away and set $d_{k+1} = -g_{k+1}$.

Nonlinear conjugate gradient has thin theoretical guarantees

d_{k+1} may not be a descent direction, so implementations **restart**: throw the direction away and set $d_{k+1} = -g_{k+1}$.

A restart test with guarantees

Given $\sigma \in (0, 1]$, $\kappa \geq 1$, $p, q \geq 0$, keep d_{k+1} only if

$$\underbrace{g_{k+1}^\top d_{k+1} \leq -\sigma \|g_{k+1}\|^{1+p}}_{\text{the direction descends}} \quad \text{and} \quad \underbrace{\|d_{k+1}\| \leq \kappa \|g_{k+1}\|^q}_{\text{its norm stays controlled}}$$

This gives the first worst-case complexity for NCG in the nonconvex setting, but only to an ε_g -FOSP.

No NCG method is known to reach an $(\varepsilon_g, \varepsilon_H)$ -SOSP.

1. How do we escape a saddle point?
2. Conjugate gradient and heavy-ball
3. The method and the result
4. Does it work?
5. Conclusion

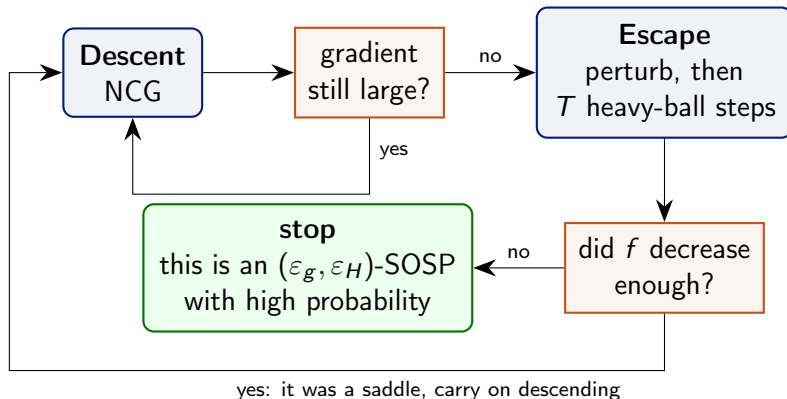
Our proposal: a two phase method

A **descent phase** reaches a FOSP, which might be a saddle point, and an **escape phase** tries to escape it.

	what it requires	what provides it
Descent	fast progress while the gradient is large ($\ g_k\ \geq \varepsilon_g$)	NCG
Escape	fixed parameters inside any given run (needed for proof technique)	heavy-ball

Algorithm diagram

Our algorithm **NCG-HB** is:



The decrease in f over one escape window is the **certificate**: no decrease means no negative curvature, so no escape was possible.

Theorem

Under Lipschitz gradient and Lipschitz Hessian, the method stops and returns a point that is an $(\varepsilon_g, \varepsilon_H)$ -SOSP with probability at least $1 - \delta$, after at most

$$\tilde{O}\left(\underbrace{\varepsilon_g^{-2} + \varepsilon_g^{-\max\{1+p, 2(1+p-q)\}}}_{\text{descent phase}} + \underbrace{\varepsilon_H^{-4}}_{\text{escape phase}}\right) \quad \text{iterations.}$$

Theorem

Under Lipschitz gradient and Lipschitz Hessian, the method stops and returns a point that is an $(\varepsilon_g, \varepsilon_H)$ -SOSP with probability at least $1 - \delta$, after at most

$$\tilde{O}\left(\underbrace{\varepsilon_g^{-2} + \varepsilon_g^{-\max\{1+p, 2(1+p-q)\}}}_{\text{descent phase}} + \underbrace{\varepsilon_H^{-4}}_{\text{escape phase}}\right) \text{ iterations.}$$

- p, q are the parameters of the restart test, typically in $[0, 1]$. The ε_g^{-2} is only for **restart steps**; accepted NCG steps correspond to the second rate, which is $\varepsilon_g^{-3/2}$ for $p = \frac{1}{2}$, $q = 1$.

Theorem

Under Lipschitz gradient and Lipschitz Hessian, the method stops and returns a point that is an $(\varepsilon_g, \varepsilon_H)$ -SOSP with probability at least $1 - \delta$, after at most

$$\tilde{O}\left(\underbrace{\varepsilon_g^{-2} + \varepsilon_g^{-\max\{1+p, 2(1+p-q)\}}}_{\text{descent phase}} + \underbrace{\varepsilon_H^{-4}}_{\text{escape phase}}\right) \quad \text{iterations.}$$

- p, q are the parameters of the restart test, typically in $[0, 1]$. The ε_g^{-2} is only for **restart steps**; accepted NCG steps correspond to the second rate, which is $\varepsilon_g^{-3/2}$ for $p = \frac{1}{2}$, $q = 1$.
- With the usual coupling $\varepsilon_H = \sqrt{\varepsilon_g}$, the escape term is ε_g^{-2} : **the rate of PGD**.

Theorem

Under Lipschitz gradient and Lipschitz Hessian, the method stops and returns a point that is an $(\varepsilon_g, \varepsilon_H)$ -SOSP with probability at least $1 - \delta$, after at most

$$\tilde{O}\left(\underbrace{\varepsilon_g^{-2} + \varepsilon_g^{-\max\{1+p, 2(1+p-q)\}}}_{\text{descent phase}} + \underbrace{\varepsilon_H^{-4}}_{\text{escape phase}}\right) \quad \text{iterations.}$$

- p, q are the parameters of the restart test, typically in $[0, 1]$. The ε_g^{-2} is only for **restart steps**; accepted NCG steps correspond to the second rate, which is $\varepsilon_g^{-3/2}$ for $p = \frac{1}{2}$, $q = 1$.
- With the usual coupling $\varepsilon_H = \sqrt{\varepsilon_g}$, the escape term is ε_g^{-2} : **the rate of PGD**.
- First second-order guarantee for a method based on NCG.

1. How do we escape a saddle point?
2. Conjugate gradient and heavy-ball
3. The method and the result
4. Does it work?
5. Conclusion

Two test problems

A quadratic saddle

$$f(x) = \frac{1}{2}x^\top Hx + \frac{\nu}{4}\|x\|^4$$

Spectrum chosen by hand, so the curvature at the saddle is known exactly.

Matrix factorization

$$f(U) = \frac{1}{4}\|UU^\top - M\|_F^2$$

Every local minimum global, every saddle strict. The origin is a saddle.

- All methods share the same stepsize and the same escape budget. Nothing is tuned.
- Escape runs **start on a saddle**: from a random point one is essentially never met, and there would be nothing to measure.
- Two variants of NCG-HB are compared against GD, PGD, PAGD.
- All numbers are medians over 20 repetitions.

The descent phase is fast

ε_g	quadratic saddle				matrix factorization			
	10^{-2}	10^{-4}	10^{-6}	10^{-8}	10^{-2}	10^{-4}	10^{-6}	10^{-8}
GD	109	144	174	204	1594	1989	2469	3234
NCG	4	15	20	26	20	26	35	47

- Iterations to reach a small gradient. **47** against 3234.
- The method never restarts here: these are real NCG steps.

method	quadratic saddle		matrix factorization	
	iterations	evaluations	iterations	evaluations
GD	never	—	6248	6249
perturbed NCG	175	181	970	1015
PGD	210	213	2181	2187
PAGD	149	596	570	2280
NCG-HB	99	105	956	967
NCG-HB, with early exit	50	68	137	203

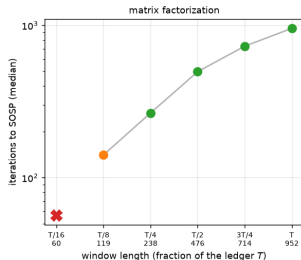
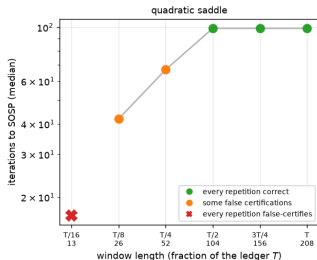
method	quadratic saddle		matrix factorization	
	iterations	evaluations	iterations	evaluations
GD	never	—	6248	6249
perturbed NCG	175	181	970	1015
PGD	210	213	2181	2187
PAGD	149	596	570	2280
NCG-HB	99	105	956	967
NCG-HB, with early exit	50	68	137	203

- In iterations: NCG-HB ahead of PGD on both, ahead of the accelerated version on one.

method	quadratic saddle		matrix factorization	
	iterations	evaluations	iterations	evaluations
GD	never	—	6248	6249
perturbed NCG	175	181	970	1015
PGD	210	213	2181	2187
PAGD	149	596	570	2280
NCG-HB	99	105	956	967
NCG-HB, with early exit	50	68	137	203

- In iterations: NCG-HB ahead of PGD on both, ahead of the accelerated version on one.
- In **evaluations of f and ∇f** ahead of both, on both. One accelerated iteration costs four evaluations; one window step costs one.

The early exit version

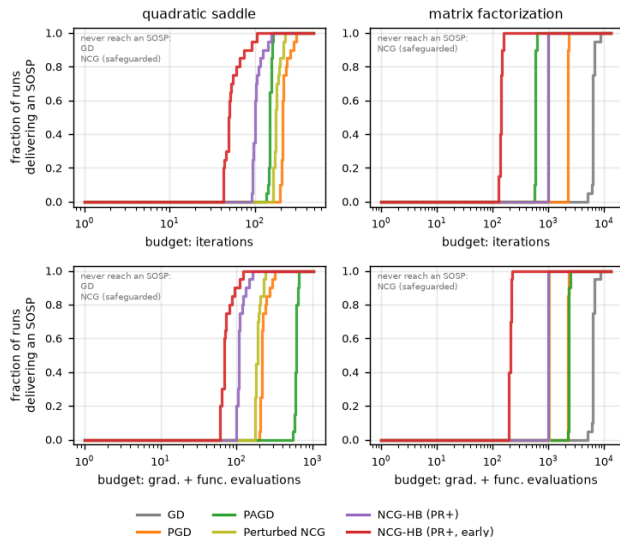


The escape window is sized for the worst case. In practice the escape is over after a fraction of its window's size, the rest is spent waiting for the budget to run out.

An **early exit** leaves the escape phase as soon as the sufficient drop in f has been observed.

Reaching a SOSP for matrix factorization: $956 \rightarrow 137$ iterations.

Convergence comparison



Our method with early exit beats the others in all configuration.

PAGD degrades badly when the budget is in evaluations.

1. How do we escape a saddle point?
2. Conjugate gradient and heavy-ball
3. The method and the result
4. Does it work?
5. Conclusion

- We derive a **second-order complexity guarantee** for an NCG-based method for the first time.
- The worst-case bound matches that of PGD, but can improve in certain regimes.
- Empirically the method reaches good points in fewer evaluations than both PGD and PAGD, on both test problems.
- In the last chapter of the thesis, we report complexity guarantees for convergence to a **local minimum** under an additional assumption on the landscape.

Short term

- Does NCG itself escape saddle points? Not answered here: it is used only to descend.
- The two phases are independent. Any descent method, any escape mechanism: which pairs work best, and does it depend on the problem?

Short term

- Does NCG itself escape saddle points? Not answered here: it is used only to descend.
- The two phases are independent. Any descent method, any escape mechanism: which pairs work best, and does it depend on the problem?

Longer term

This becomes the first chapter of a PhD, on exploiting landscape structure in nonconvex optimization, with applications to scientific imaging and inverse problems.

Thank you.

Questions

Closest related work

- R. Chan–Renous–Legoubin and C. W. Royer. *A nonlinear conjugate gradient method with complexity guarantees and its application to nonconvex regression*. EJCO, 2022.
- C. Jin, R. Ge, P. Netrapalli, S. M. Kakade and M. I. Jordan. *How to escape saddle points efficiently*. ICML, 2017.
- C. Jin, P. Netrapalli and M. I. Jordan. *Accelerated gradient descent escapes saddle points faster than gradient descent*. COLT, 2018.
- H. Li and Z. Lin. *Restarted nonconvex accelerated gradient descent: no more polylogarithmic factor in the $O(\epsilon^{-7/4})$ complexity*. JMLR, 2023.
- M. O’Neill and S. J. Wright. *Behavior of accelerated gradient methods near critical points of nonconvex functions*. Math. Program., 2019.

Saddle points and landscapes

- K. G. Murty and S. N. Kabadi. *Some NP-complete problems in quadratic and nonlinear programming*. Math. Program., 1987.

- Y. N. Dauphin, R. Pascanu, C. Gulcehre, K. Cho, S. Ganguli and Y. Bengio. *Identifying and attacking the saddle point problem in high-dimensional nonconvex optimization*. NeurIPS, 2014.
- R. Ge, F. Huang, C. Jin and Y. Yuan. *Escaping from saddle points — online stochastic gradient for tensor decomposition*. COLT, 2015.
- J. D. Lee, M. Simchowitz, M. I. Jordan and B. Recht. *Gradient descent only converges to minimizers*. COLT, 2016.
- J. D. Lee, I. Panageas, G. Piliouras, M. Simchowitz, M. I. Jordan and B. Recht. *First-order methods almost always avoid strict saddle points*. Math. Program., 2019.
- S. Du, C. Jin, J. D. Lee, M. I. Jordan, A. Singh and B. Póczos. *Gradient descent can take exponential time to escape saddle points*. NeurIPS, 2017.
- A.-A. Muşat and N. Boumal. *Gradient descent avoids strict saddles with a simple line-search method too*. Preprint, 2025.

Second-order and hybrid methods

- Y. Nesterov and B. T. Polyak. *Cubic regularization of Newton method and its global performance*. Math. Program., 2006.
- C. Cartis, N. I. M. Gould and P. L. Toint. *Adaptive cubic regularisation methods for unconstrained optimization. Part I*. Math. Program., 2011.
- C. W. Royer, M. O'Neill and S. J. Wright. *A Newton-CG algorithm with complexity guarantees for smooth unconstrained optimization*. Math. Program., 2020.
- F. E. Curtis, D. P. Robinson, C. W. Royer and S. J. Wright. *Trust-region Newton-CG with strong second-order complexity guarantees for nonconvex optimization*. SIAM J. Optim., 2021.

Conjugate gradient and momentum

- B. T. Polyak. *Some methods of speeding up the convergence of iteration methods*. USSR Comput. Math. and Math. Phys., 1964.
- R. Fletcher and C. M. Reeves. *Function minimization by conjugate gradients*. The Computer Journal, 1964.
- E. Polak and G. Ribière. *Note sur la convergence de méthodes de directions conjuguées*. R.I.R.O., 1969.

Figures

- J. Wright and Y. Ma. *High-Dimensional Data Analysis with Low-Dimensional Models*. Cambridge University Press, 2022.